P1

Resumo Foco Prova — Redes de Computadores (UFRJ)

> Versão enxuta, construída a partir do resumo completo + análise das 6 provas antigas fornecidas (2022/1, 2022/2, 2024/2, 2025/1, 2025/2, 2026/1). Mantém apenas os temas que **de fato caíram** nessas provas, removendo tópicos que nunca apareceram (histórico da Internet, camadas/encapsulamento, segurança, CDN/DASH, BitTorrent tit-for-tat, código Python de sockets, correio eletrônico). Se precisar de algum desses temas removidos, é só pedir — o resumo completo ainda está disponível.

PARTE I — TEORIA (temas de alta recorrência)

1.3 O Núcleo da Rede (*Network Core*)

O núcleo da rede é a malha interconectada de comutadores de pacotes (*roteadores* e *switches*) e enlaces que conectam as redes de acesso entre si. Sua função primária é transportar dados de um sistema final de origem para um de destino através de nós intermediários.

1.3.1 Comutação de Pacotes (*Packet Switching*)

Conceito

Na comutação de pacotes, os sistemas finais dividem as mensagens em blocos discretos de dados chamados **pacotes**. Cada pacote atravessa a rede transitando por enlaces e comutadores de pacotes. Os dados são transmitidos à taxa total do enlace de saída.

1. Transmissão Armazena-e-Reenvia (*Store-and-Forward*)

A maioria dos comutadores de pacotes opera segundo o princípio de **armazena-e-reenvia**: o comutador precisa receber o pacote por inteiro em seu *buffer* de entrada antes de poder começar a transmitir o primeiro bit do pacote para o enlace de saída.

- **Análise de Atraso Simples:** Se enviarmos 1 pacote de (L) bits através de (N) enlaces idênticos em série, cada um com taxa de transmissão (R) (e ignorando a propagação por enquanto), o atraso fim a fim é: $d_{\text{fim-a-fim}} = N \cdot \frac{L}{R}$

2. Atraso de Fila e Perda de Pacotes

Cada comutador possui múltiplos enlaces de saída. Para cada enlace, o comutador mantém um **buffer de saída** (ou fila):

- **Formação de Fila:** Se um pacote chega e encontra o enlace ocupado transmitindo outro pacote, ele precisa aguardar no *buffer*.
- **Perda de Pacote (*Packet Loss*):** Como a memória do *buffer* é finita, se um pacote chegar e encontrar o *buffer* completamente cheio, o comutador não terá onde armazená-lo e o descartará (*drop*).

3. Tabelas de Repasse e Protocolos de Roteamento

Como o roteador sabe para qual enlace de saída enviar determinado pacote?

- Cada pacote contém em seu cabeçalho o **endereço IP de destino**.
- O roteador possui uma **tabela de repasse** (*forwarding table*) que mapeia endereços IP de destino para seus enlaces de saída físicos.
- As tabelas de repasse são construídas automaticamente por **protocolos de roteamento** (como OSPF e BGP) que calculam os caminhos mais curtos e eficientes através da malha.

1.3.2 Comutação de Circuitos (*Circuit Switching*)

Conceito

Na comutação de circuitos, os recursos necessários ao longo de um caminho para prover a comunicação entre os sistemas finais (largura de banda, tempos de comutação) são **reservados e dedicados exclusivamente** durante toda a duração da sessão de comunicação.

```
[ Usuário A ]  ===(Recursos Reservados/Dedicados)===  [ Usuário B ]
```

Multiplexação na Comutação de Circuitos

Para permitir que múltiplos circuitos compartilhem o mesmo cabo físico, utilizam-se duas técnicas clássicas de multiplexação:

1. **FDM (*Frequency-Division Multiplexing*):** O espectro de frequência do meio é dividido em faixas fixas. Cada conexão recebe sua própria faixa dedicada.
2. **TDM (*Time-Division Multiplexing*):** O tempo é dividido em quadros (*frames*) com número fixo de compartimentos de tempo (*slots*). Cada conexão recebe um *slot* exclusivo por quadro.

1.3.3 Comparação e Unificação Matemática: Multiplexação Estatística (Kurose + UFRJ)

Fenômeno de Rede

Por que a Internet moderna utiliza comutação de pacotes em vez de comutação de circuitos? Os usuários de rede não transmitem dados continuamente; o tráfego de aplicações Web, e-mail e áudio/vídeo é essencialmente **em rajadas** (*bursty*) — alternando períodos curtos de transmissão intensa com longos períodos de ociosidade.

Unificação Matemática 1: O Modelo Binomial de Usuários Ativos

1. O que representa?

Modela o número aleatório de usuários ativamente transmitindo dados ao mesmo tempo em um enlace compartilhado por uma população finita de usuários independentes.

2. Por que estamos usando esse modelo?

Para provar quantitativamente por que a comutação de pacotes permite admitir **muito mais usuários** na mesma infraestrutura do que a comutação de circuitos, mantendo a probabilidade de congestionamento (*sobrecarga*) extremamente baixa.

3. Significado das Variáveis e Unidades

- m (ou N): Número total de usuários independentes que compartilham o enlace.
- p : Probabilidade de um usuário individual estar ativo (transmitindo dados) em um dado instante.
- A (ou X): Variável aleatória discreta que representa o **número de usuários ativos simultaneamente** em um dado instante.
- R : Capacidade total do enlace de saída (ex: $1\ \mathrm{Mbps}$).
- (r) (ou C): Número máximo de usuários simultâneos que a rede consegue atender sem formar fila (ex: se cada usuário precisa de $100\ \mathrm{kbps}$, um enlace de $1\ \mathrm{Mbps}$ suporta $r = \frac{1\ \mathrm{Mbps}}{100\ \mathrm{kbps}} = 10$ usuários dedicados).

4. Hipóteses do Modelo

- Os m usuários comportam-se de maneira totalmente **independente**.
- Cada usuário tem a mesma probabilidade p de estar ativo em qualquer instante.

5. Aplicação da Fórmula

Como cada usuário está ativo (1) com probabilidade p ou inativo (0) com probabilidade 1-p , o número total de ativos A segue uma **Distribuição Binomial**:

$$A \sim \mathrm{Binomial}(m, p)$$

A probabilidade de haver exatamente a usuários ativos no mesmo instante é dada pela Função de Massa de Probabilidade (PMF):

$$P(A = a) = \binom{m}{a} p^a (1-p)^{m-a} = \frac{m!}{a!(m-a)!} p^a (1-p)^{m-a}$$

A **Carga Esperada** (número médio de usuários ativos) é:

$$E[A] = m \cdot p$$

- **Em Comutação de Circuitos:** Admitimos no máximo m = r usuários. Se r=10 , apenas 10 usuários podem ser aceitos, mesmo que passem a maior parte do tempo em silêncio.
- **Em Comutação de Pacotes:** Permitimos que m > r usuários compartilhem o enlace. Haverá **sobrecarga/congestionamento** quando A > r . A probabilidade de sobrecarga P(A > r) é a cauda da distribuição binomial:

$$P(A > r) = \sum_{a=r+1}^{m} \binom{m}{a} p^a (1-p)^{m-a} = 1 - \sum_{a=0}^{r} \binom{m}{a} p^a (1-p)^{m-a}$$

6. Cálculo e Interpretação (Exemplo Clássico Kurose / Prova UFRJ)

Considere um enlace de $1\ \mathrm{Mbps}$, onde cada usuário consome $100\ \mathrm{kbps}$ quando ativo, mas fica ativo apenas durante 10%) do tempo ($p = 0{,}1$).

1. **Comutação de Circuitos:** Cada usuário requer $100\ \mathrm{kbps}$ reservados. O enlace suporta estritamente r = 10 usuários. O 11º usuário é bloqueado.
2. **Comutação de Pacotes:** Conectamos **m = 35 usuários** no mesmo enlace!
   - Carga média esperada: E[A] = $35 \times 0{,}1 = 3{,}5$ usuários ativos em média.
   - Probabilidade de sobrecarga (mais de 10 usuários ativos simultaneamente): $P(A > 10)$ = $\sum_{a=11}^{35} \binom{35}{a} (0{,}1)^a (0{,}9)^{35-a} \approx 0{,}0004 \quad (0{,}04$

- **Interpretação:** Em $99{,}96$ do tempo, há 10 ou menos usuários ativos e a rede funciona sem nenhum atraso de fila. Suportamos **3,5 vezes mais usuários** do que na comutação de circuitos com uma taxa de sobrecarga insignificante (menos de 4 vezes em 10.000 instantes). Isso é o poder da **multiplexação estatística**.

Unificação Matemática 2: A Aproximação de Poisson (UFRJ)

1. O que representa?

Modela o número de usuários ativos quando a população total de usuários (m) é muito grande, mas a probabilidade de cada um estar ativo (p) é muito pequena.

2. Por que estamos usando esse modelo?

Calcular somatórios binomiais com números combinatórios $\binom{m}{a}$ para $m = 1000$ ou mais é computacionalmente custoso e suscetível a estouros de precisão. A distribuição de Poisson fornece uma aproximação analítica rápida e precisa.

3. Significado das Variáveis e Unidades

- $\lambda = m \cdot p$: Carga média total mantida constante (unidade: número médio de usuários ativos).

4. Hipóteses

- $m \to \infty$ (grande número de fontes).
- $p \to 0$ (cada fonte raramente ativa).
- $\lambda = m \cdot p$ permanece moderado.

5. Aplicação da Fórmula

Substituindo o limite da Binomial quando $m \to \infty$, obtemos a PMF da **Poisson**:

$$P(A = a) \approx \frac{e^{-\lambda} \cdot \lambda^a}{a!}$$

Unificação Matemática 3: O Tempo de Descongestionamento — Modelo Geométrico (Notas UFRJ)

1. O que representa?

Modela o número de *slots* de tempo necessários (S) para que a rede saia do estado de sobrecarga e volte a ficar descongestionada.

2. Por que estamos usando esse modelo?

A variável binomial A tira apenas uma "fotografia" estática de um instante da rede. A variável temporal S responde à pergunta do operador de rede: *"Dado que a rede congestionou no slot atual, quantos slots levaremos, em média, até a rede voltar ao normal?"*

3. Significado das Variáveis e Unidades

- q : Probabilidade de um *slot* estar descongestionado (sucesso, ou seja, $P(A \leq r)$).
- 1 - q : Probabilidade de o *slot* estar congestionado ($P(A > r)$).
- S : Número de *slots* futuros até observar o primeiro *slot* descongestionado ($S \in 1, 2, 3, \ldots$).

4. Hipóteses

- Os estados da rede em *slots* consecutivos são temporalmente **independentes**.

5. Aplicação das Fórmulas

Como estamos contando o número de tentativas até obter o primeiro sucesso (slot descongestionado), S segue uma **Distribuição Geométrica**:

$$S \sim \text{Geom}(q)$$

$$P(S = s) = (1 - q)^{s-1} \cdot q, \quad s = 1, 2, 3, \ldots$$

O **Tempo Médio de Recuperação (Esperança de S)** é:

$$E[S] = \frac{1}{q}$$

6. Interpretação em Termos de Redes

Se a probabilidade de um slot estar normal for $q = 0{,}50$, o tempo médio de descongestionamento é $E[S] = \frac{1}{0{,}50} = 2 \text{ slots}$. Se a rede estiver extremamente sobrecarregada com $q = 0{,}20$, levará em média $E[S] = \frac{1}{0{,}20} = 5 \text{ slots}$ para se recuperar.

1.4 Atraso, Perda e Vazão em Redes de Comutação de Pacotes

1.4.1 Visão Geral dos Atrasos Nodal e Fim a Fim

Fenômeno de Rede

Quando um pacote transita da origem ao destino, ele passa por uma sequência de roteadores intermediários. Em cada nó, o pacote não é repassado instantaneamente; ele sofre uma série de atrasos físicos e operacionais.

Conceito

O **Atraso Nodal Total** (d_{nodal}) em um único comutador de pacotes é a soma de quatro componentes fundamentais:

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{fila}} + d_{\text{trans}} + d_{\text{prop}}$$

```
[ Chegada ] ---> [ Processamento (d_proc) ] ---> [ Fila de Espera (d_fila) ] ---> [
Transmissão (d_trans) ] ---> [ Enlace (d_prop) ] ---> [ Próximo Nó ]
```

1. **Atraso de Processamento (d_{proc}):** Tempo que a CPU do roteador leva para examinar o cabeçalho do pacote, verificar erros de *checksum* e determinar a interface de saída pela tabela de repasse. Tipicamente na ordem de **microssegundos** ou menos.
2. **Atraso de Fila (d_{fila} ou W):** Tempo que o pacote passa aguardando no *buffer* de saída enquanto outros pacotes à sua frente são transmitidos. Varia dinamicamente de 0 até milissegundos dependendo do tráfego.
3. **Atraso de Transmissão (d_{trans} ou S):** Tempo para injetar (serializar) todos os L bits do pacote no enlace. $d_{\text{trans}} = L/R$.
4. **Atraso de Propagação (d_{prop}):** Tempo para um único bit percorrer a distância física do meio à velocidade da luz no meio. $d_{\text{prop}} = d/s$.

1.4.2 Atraso de Fila e Intensidade de Tráfego

Fenômeno de Rede

Por que a fila se forma? Se a taxa na qual os bits chegam à porta de saída do roteador for maior do que a taxa na qual o enlace consegue transmiti-los, os pacotes precisam ser armazenados temporariamente no *buffer*.

Conceito

A métrica fundamental para avaliar o estado de congestionamento de um nó é a **Intensidade de Tráfego** (I, também denotada por ρ em teoria de filas).

Unificação Matemática 1: Intensidade de Tráfego e Condição de Estabilidade

1. O que representa?

A razão entre a taxa média de chegada de bits e a capacidade de transmissão do enlace. Representa também a **utilização do enlace** (U), ou seja, a fração de tempo em que o enlace fica ocupado transmitindo dados.

2. Por que estamos usando este modelo?

Para estabelecer o limite físico sob o qual uma rede opera de forma estável.

3. Significado das Variáveis e Unidades

- a (ou λ): Taxa média de chegada de pacotes ao nó ($\text{pacotes}/\text{segundo}$).
- L: Tamanho médio dos pacotes (bits).
- R: Taxa de transmissão do enlace (bits/s).
- $\mu = \frac{R}{L}$: Taxa média de serviço do enlace ($\text{pacotes}/\text{segundo}$).
- $S = \frac{1}{\mu} = \frac{L}{R}$: Tempo médio de serviço/transmissão de um pacote (segundos).

4. Hipóteses

- O sistema opera em **equilíbrio estacionário** (o que entra sai).
- O *buffer* possui capacidade ilimitada para armazenar pacotes.

5. Aplicação da Fórmula

$$I = \rho = \frac{a \cdot L}{R} = \frac{\lambda}{\mu} \quad [\text{adimensional}]$$

- **Se $I > 1$:** Chega mais trabalho do que o enlace pode processar. A fila cresce sem limite e o atraso tende ao infinito ($d_{\text{fila}} \to \infty$).
- **Se $I < 1$:** O sistema é estável. A utilização do servidor é $U = I$.
- **Se $I = 1$:** O sistema só é estável se as chegadas e os tempos de serviço forem estritamente **determinísticos** ($D/D/1$) sem nenhuma variabilidade. Na presença de aleatoriedade, $I = 1$ causa estouro de fila.

Unificação Matemática 2: A Lei de Little (UFRJ)

1. O que representa?

Uma lei fundamental e universal da teoria de filas que relaciona a quantidade média de entidades no sistema com a taxa de entrada e o tempo médio de permanência.

2. Por que estamos usando este modelo?

Porque ela **não depende** da distribuição probabilística do tráfego nem da regra de atendimento; ela se aplica a qualquer sistema em equilíbrio.

3. Significado das Variáveis e Unidades

- a (ou λ): Taxa média efetiva de chegada de pacotes ($\text{pacotes}/\text{s}$).
- T: Tempo total médio que um pacote passa no sistema (fila + serviço) (s).
- W (ou d_{fila}): Tempo médio de espera do pacote estritamente na **fila** (s).
- $S = \frac{L}{R}$: Tempo médio de transmissão no **servidor** (s).
- N: Número médio de pacotes no **sistema completo** ($N = N_q + N_s$).
- N_q (ou $E[B]$): Número médio de pacotes aguardando na **fila**.
- N_s: Número médio de pacotes em transmissão no **servidor** ($N_s = U = I$).

4. Hipóteses

- O sistema está em equilíbrio estatístico e conservação de taxa ($I < 1$).

5. Aplicação das Fórmulas de Little

1. **Ao Sistema Completo:** $N = a \cdot T$
2. **Apenas à Fila de Espera:** $N_q = a \cdot W$
3. **Apenas ao Servidor:** $N_s = a \cdot S = a \cdot \frac{L}{R} = I$

Unificação Matemática 3: O Modelo de Fila $M/M/1$ (Gabarito Provas UFRJ)

1. O que representa?

Modela o atraso e a ocupação de um buffer de roteador assumindo **chegadas de Poisson** (M) e **tempos de transmissão exponenciais** (M, ou seja, tamanhos de pacote aleatórios).

2. Por que estamos usando este modelo?

Fornece uma expressão analítica exata para prever como o atraso de fila cresce de forma assintótica conforme a utilização I se aproxima de 1.

3. Hipóteses do Modelo

- **Chegadas de Poisson** (propriedade **PASTA**: *Poisson Arrivals See Time Averages* — o pacote que chega enxerga a média temporal do estado da rede).
- **Tempos de serviço exponenciais** (propriedade de **falta de memória** — o tempo residual de transmissão do pacote atual é em média igual ao tempo total de serviço S).

4. Derivação Passo a Passo (Exigida nas Provas da UFRJ)

Um pacote recém-chegado encontra, em média, N_q pacotes na fila e $N_s = I$ pacotes no servidor. Pela falta de memória e pela propriedade PASTA, seu tempo médio de espera na fila W é:

$$W = (N_q + N_s) \cdot S = (N_q + I) \cdot \frac{L}{R}$$

Substituindo a Lei de Little $N_q = a \cdot W$:

$$W = (a \cdot W + I) \cdot \frac{L}{R} = a \cdot W \cdot \frac{L}{R} + I \cdot \frac{L}{R} = I \cdot W + I \cdot \frac{L}{R}$$

Isolando W:

$$W(1 - I) = I \cdot \frac{L}{R} \quad \Longrightarrow \quad W_{M/M/1} = \frac{I}{1 - I} \cdot \frac{L}{R} = \frac{I}{\mu(1 - I)}$$

Aplicando as demais métricas pela Lei de Little:

- **Tempo Total no Sistema (T):**

$$T = W + S = \frac{I}{1 - I}\frac{L}{R} + \frac{L}{R} = \frac{1}{1 - I} \cdot \frac{L}{R} = \frac{1}{\mu - a}$$

- **Número Médio de Pacotes no Sistema (N):**

$$N = a \cdot T = \frac{a}{\mu - a} = \frac{I}{1 - I}$$

- **Número Médio de Pacotes na Fila (N_q):**

$$N_q = a \cdot W = \frac{I^2}{1 - I}$$

Unificação Matemática 4: O Modelo de Fila $M/D/1$ (Pacotes Fixos)

1. O que representa?

Modela a fila em um roteador onde todos os pacotes têm **tamanho fixo/determinístico** de L bits (serviço D).

2. Por que estamos usando este modelo?

Para demonstrar o **efeito da variabilidade do tamanho dos pacotes** no atraso da rede.

3. Hipóteses e Aplicação da Fórmula

Como todos os pacotes têm exatamente o mesmo tamanho, o tempo residual de transmissão que um pacote recém-chegado encontra no servidor é, em média, **metade do tempo de transmissão** ($S_R = \frac{S}{2} = \frac{L}{2R}$).

Repetindo a dedução da UFRJ com tempo residual $S/2$:

$$W_{M/D/1} = N_q \cdot S + I \cdot \frac{S}{2} = aW \cdot S + \frac{I \cdot S}{2} = I \cdot W + \frac{I \cdot S}{2}$$

$$W_{M/D/1}(1 - I) = \frac{I \cdot S}{2} \quad \Longrightarrow \quad W_{M/D/1} = \frac{I}{2(1-I)} \cdot \frac{L}{R} = \frac{1}{2} W_{M/M/1}$$

6. Interpretação em Termos de Redes

$$W_{M/D/1} = \frac{1}{2} W_{M/M/1}$$

Padronizar o tamanho dos pacotes na rede (tornando-os determinísticos) **reduz o tempo de espera na fila exatamente pela metade** em relação a uma rede com tamanhos de pacotes altamente variáveis ($M/M/1$).

1.4.3 Perda de Pacotes (*Packet Loss*)

Fenômeno de Rede

Na prática, a memória do *buffer* de um roteador é **finita**. O que acontece quando chegam pacotes e a fila já está 100% cheia?

Conceito

Como não há espaço físico para armazenar o pacote recebido, o roteador simplesmente o **descarta** (*packet drop*).

- **Do ponto de vista do sistema final:** O pacote foi transmitido, mas "sumiu" no núcleo da rede.
- A fração de pacotes perdidos aumenta exponencialmente conforme a intensidade de tráfego I se aproxima ou excede 1.
- Protocolos da camada de transporte confiável (como o TCP) precisam detectar essa perda e retransmitir o pacote, gerando atraso adicional para a aplicação.

1.4.4 Atraso Fim a Fim (*End-to-End Delay*)

Acumulando os atrasos nodais ao longo de um caminho com $N-1$ roteadores intermediários ($H = N$ enlaces idênticos) sem congestionamento:

$$d_{\text{fim-a-fim}} = N \cdot (d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}}) = N \left(d_{\text{proc}} + \frac{L}{R} + \frac{d}{s} \right)$$

Ferramenta Prática de Diagnóstico: Traceroute

O programa **Traceroute** permite medir o atraso de ida e volta (RTT) real até cada roteador ao longo do caminho enviando múltiplos pacotes com valores crescentes de TTL (*Time-To-Live*). Se um roteador não responder ou descartar pacotes devido a congestionamento, o Traceroute exibe um asterisco (`*`).

1.4.5 Vazão em Redes de Computadores (*Throughput*)

Fenômeno de Rede

A qual taxa efetiva (em bits por segundo) um arquivo é transferido da origem ao destino através de uma sequência de enlaces compartilhados?

Conceito e O Enlace Gargalo (*Bottleneck Link*)

- **Vazão Instantânea:** A taxa em um dado instante de tempo.
- **Vazão Média:** Se a transferência de um arquivo de F bits leva T segundos, a vazão média é F/T bits/s.

Modelo Matemático: A Vazão em Enlaces em Série e Compartilhados

1. Aplicação da Fórmula

Considere um caminho com N enlaces em série com taxas de transmissão $R_1, R_2, \ldots, R_N$. A vazão fim a fim $R_{\text{vazão}}$ é dada estritamente pela taxa do **enlace mais lento** do caminho (o **gargalo**):

$$R_{\text{vazão}} = \min R_1, R_2, \ldots, R_N$$

```
[ Servidor ] ---( R_1 = 100 Mbps )---> [ Roteador ] ---( R_2 = 1.5 Mbps )---> [
Roteador ] ---( R_3 = 10 Mbps )---> [ Cliente ]
                                                          ↑
                                                ENLACE GARGALO (Vazão = 1.5 Mbps)
```

2. Efeito de Enlaces Compartilhados e Caching (Notas UFRJ)

Se o enlace gargalo de taxa R for compartilhado por K conexões simultâneas, a vazão de cada conexão é $\min\left(R_s, R_c, \frac{R}{K}\right)$.

Ao instalar um **servidor Cache (Web Cache)** próximo aos clientes com uma taxa de acerto (*hit rate*) h, reduzimos a taxa de chegada no enlace de acesso de a para uma taxa efetiva a_{efetiva}:

$$a_{\text{efetiva}} = (1 - h) \cdot a$$

Isso reduz drasticamente a intensidade de tráfego $I = \frac{L \cdot a_{\text{efetiva}}}{R}$, retirando o enlace da zona de congestionamento e reduzindo o atraso de fila de segundos para milissegundos!

2.1.3 Requisitos de Transporte Exigidos pelas Aplicações

Diferentes aplicações possuem necessidades distintas quanto ao serviço fornecido pela camada de transporte:

1. **Confiabilidade e Integridade de Dados (*Data Loss*):**
   - *Aplicações tolerantes a perdas:* Áudio e vídeo em tempo real ou jogos *online* toleram uma pequena porcentagem de pacotes perdidos sem comprometer criticamente a experiência do usuário.
   - *Aplicações sensíveis à perda:* Transferência de arquivos (FTP), e-mail, transações bancárias e texto Web exigem entrega 100% confiável e sem erros.
2. **Vazão (*Throughput*):**
   - *Aplicações sensíveis à largura de banda: Streaming* de vídeo em alta definição exige taxas de bits mínimas garantidas para evitar travamentos.
   - *Aplicações elásticas:* E-mail, navegação Web e transferência de arquivos utilizam a largura de banda que estiver disponível no momento.
3. **Temporização (*Timing* / Latência):**
   - Aplicações interativas em tempo real (telefonia IP, jogos multijogador) exigem garantias de baixos atrasos fim a fim (por exemplo, entrega de bits em menos de 100 ms) para evitar pausas artificiais na conversação.
4. **Segurança:**
   - Criptografia de dados de ponta a ponta, integridade de mensagem e autenticação das partes comunicantes.

2.1.4 Serviços de Transporte Fornecidos pela Internet: TCP vs. UDP

A camada de transporte da Internet oferece dois protocolos principais para as aplicações:

PROPRIEDADE DE TRANSPORTE	TCP (*TRANSMISSION CONTROL PROTOCOL*)	UDP (*USER DATAGRAM PROTOCOL*)
Orientação à Conexão	**Orientado à Conexão**: Exige o *handshake* de três vias antes da troca de dados.	**Sem Conexão (*Stateless*)**: Envia datagramas sem estabelecimento de conexão prévia.

PROPRIEDADE DE TRANSPORTE	TCP (*TRANSMISSION CONTROL PROTOCOL*)	UDP (*USER DATAGRAM PROTOCOL*)
Confiabilidade	**Garantida**: Entrega ordenada e sem perdas (*Byte-stream*) por ACKs e retransmissões.	**Não Confiável**: Pacotes podem ser perdidos, duplicados ou chegar fora de ordem.
Controle de Fluxo/Congestionamento	**Sim**: Adapta a taxa de envio para não afogar o receptor nem sobrecarregar os roteadores.	**Não**: Envia na taxa que a aplicação injetar no socket.
Garantia de Vazão/Delay	**Não**: Não oferece garantias mínimas de banda ou limite máximo de atraso.	**Não**: Não oferece garantias mínimas de banda ou limite máximo de atraso.
Sobrecarga (*Overhead*)	Maior (cabeçalho de 20 bytes + latência de *handshake*).	Mínima (cabeçalho leve de 8 bytes, transmissão imediata).
Uso Típico	Web (HTTP), E-mail (SMTP), Transferência de Arquivos.	DNS (consultas rápidas), Streaming de Vídeo/Áudio em Tempo Real, VoIP.

2.2 A Web e o HTTP

---

1. O Conceito de HTTP e a Semântica Sem Estado (*Stateless*)

Fenômeno de Rede

Quando um usuário navega pela Web, ele requisita arquivos HTML, imagens JPEG, arquivos CSS e scripts JS espalhados por servidores na rede.

Conceito

O **HTTP (*Hypertext Transfer Protocol*)** é o protocolo da camada de aplicação que governa a comunicação Web. Ele opera segundo a arquitetura **Cliente-Servidor**:

- **Navegador (Cliente):** Envia mensagens de requisição HTTP (*HTTP Request*) solicitando objetos.
- **Servidor Web:** Recebe a requisição, localiza o objeto e devolve uma mensagem de resposta (*HTTP Response*) contendo o objeto solicitado.

A Propriedade *Stateless* (Sem Estado)

O HTTP é um **protocolo sem estado (*stateless*)**. O servidor Web atende às requisições do cliente sem armazenar informações sobre requisições anteriores feitas por aquele mesmo usuário.

- **Vantagem:** Simplifica drasticamente o projeto do servidor, permitindo que ele atenda a milhões de requisições simultâneas sem consumir memória para manter o histórico de cada cliente.

- **Contorno da Limitação:** Para aplicações que exigem estado (como carrinhos de compras e sessões de login), a camada de aplicação utiliza **cookies** e dados de sessão.

---

2. A Definição do RTT e o *Three-Way Handshake* do TCP

Para calcular quanto tempo leva para carregar uma página Web, precisamos quantificar o tempo de viagem de um pacote de ida e volta na rede.

Modelo Matemático: O RTT e a Abertura de Conexão TCP

1. O que representa?

- **RTT (*Round-Trip Time*):** O tempo necessário para um pequeno pacote viajar do cliente ao servidor e retornar ao cliente.
- **Composição do RTT:** O RTT inclui a soma de todos os atrasos físicos do caminho: atrasos de propagação nos enlaces, atrasos de fila nos roteadores e atrasos de processamento.

2. Por que estamos usando este modelo?

O HTTP não transporta dados diretamente pelo meio físico; ele utiliza o protocolo **TCP** na camada de transporte para garantir uma entrega confiável. Antes de enviar o primeiro `GET` HTTP, o cliente precisa obrigatoriamente estabelecer uma conexão TCP via **Apresentação de Três Vias (*Three-Way Handshake*)**.

3. Aplicação do Modelo de Conexão

1. **Passo 1 (SYN):** Cliente envia um segmento TCP SYN ao servidor.
2. **Passo 2 (SYN/ACK):** Servidor responde com um segmento TCP SYN/ACK. *(Estes dois passos consomem exatos **1 RTT**).*
3. **Passo 3 (ACK + GET):** O cliente envia o ACK de confirmação combinado com a mensagem de requisição `HTTP GET`.
4. **Passo 4 (Resposta):** O servidor processa o pedido e envia o arquivo HTML/Objeto de volta. *(A requisição e a recepção do objeto consomem mais **1 RTT**).*

4. Interpretação em Termos de Latência Web

A transferência do primeiro arquivo de uma página Web (o HTML base) exige, no mínimo:

$$T_{\text{base}} = 2\,\text{RTT} + t_{\text{transmissão}}$$

onde **1 RTT** é gasto no *handshake* TCP e **1 RTT** é gasto na requisição/resposta HTTP.

---

3. Os 4 Modelos de Conexão HTTP e a Análise Matemática de Latência (Kurose + UFRJ)

Fenômeno de Rede

Uma página Web moderna raramente possui apenas um arquivo HTML; ela contém um arquivo HTML base e n **objetos referenciados** (imagens, vídeos, scripts) no mesmo servidor. Como o navegador deve organizar o transporte desses n objetos pela rede?

Combinando as opções de **Persistência** da conexão TCP com o **Paralelismo** do navegador, surgem 4 modelos clássicos de transporte analisados na UFRJ:

Modelo 1: HTTP Não Persistente Sem Paralelismo (HTTP/1.0 Padrão)

1. Funcionamento

A conexão TCP é aberta para transferir um único objeto e **imediatamente fechada** após a entrega. Os n objetos referenciados são baixados estritamente em série.

2. Análise Matemática do Tempo Total

Para o arquivo-base HTML e os n objetos referenciados:

- O HTML base custa $2\ \mathrm{RTT}$ (1 RTT para TCP + 1 RTT para HTTP).
- Cada um dos n objetos exige uma nova conexão TCP, custando $2\ \mathrm{RTT}$ cada.

$$T_{\text{não-persistente, em série}} = 2\ \mathrm{RTT} + 2n\ \mathrm{RTT} = 2(n+1)\ \mathrm{RTT}$$

(desprezando o tempo de transmissão de objetos pequenos).

3. Avaliação de Desempenho

- **Vantagens:** O servidor libera memórias e descritores de *sockets* rapidamente assim que entrega o objeto, sendo mais simples e resiliente contra ataques de conexões presas.
- **Desvantagens:** Pior desempenho de latência. O cliente paga o custo de $1\ \mathrm{RTT}$ de *handshake* repetidamente para cada imagem pequena da página.

Modelo 2: HTTP Não Persistente Com Paralelismo

1. Funcionamento

O navegador abre **múltiplas conexões TCP independentes e simultâneas** (geralmente até $k = 6$) para baixar os n objetos em paralelo.

2. Análise Matemática do Tempo Total

- O HTML base custa $2\ \mathrm{RTT}$.
- Assumindo paralelismo perfeito (onde as conexões simultâneas baixam os objetos ao mesmo tempo), os RTTs dos n objetos são sobrepostos.

$$T_{\text{não-persistente, paralelo}} = 2\ \mathrm{RTT}\ (\text{HTML}) + 2\ \mathrm{RTT}\ (\text{Objetos em paralelo}) = 4\ \mathrm{RTT}$$

3. Avaliação de Desempenho

- **Vantagens:** Redução drástica no tempo de carregamento perceptível.
- **Desvantagens:** Consumo massivo de recursos no servidor, que precisa processar múltiplos *handshakes* e manter vários *sockets* abertos simultaneamente para o mesmo cliente.

Modelo 3: HTTP Persistente Sem Paralelismo (HTTP/1.1 Padrão / *Pipelining*)

1. Funcionamento

O servidor **deixa a conexão TCP aberta** após enviar a resposta do HTML. Os n objetos subsequentes são solicitados através da **mesma conexão TCP** já estabelecida.

2. Análise Matemática do Tempo Total

- O HTML base custa $2\ \mathrm{RTT}$ (1 RTT TCP + 1 RTT HTTP).
- Como o TCP já está aberto, cada um dos n objetos subsequentes custa apenas **1 RTT** (apenas o pedido/resposta HTTP).

$$T_{\text{persistente, em série}} = 2\ \mathrm{RTT} + n\ \mathrm{RTT} = (n+2)\ \mathrm{RTT}$$

3. Avaliação de Desempenho

- **Vantagens:** Economiza $1\ \mathrm{RTT}$ de *handshake* por objeto e poupa CPU e memória do servidor ao evitar a criação/destruição contínua de *sockets*.
- **Desvantagens:** O servidor precisa gerenciar *timeouts* de conexões ociosas. Continua sujeito ao **bloqueio de cabeça de fila (*Head-of-Line Blocking*)**.

Modelo 4: HTTP Persistente Com Paralelismo (Web Moderna)

1. Funcionamento

O cliente **reaproveita a conexão TCP persistente existente** para enviar requisições sequenciais, mas **abre uma nova conexão TCP em paralelo** para acelerar objetos adicionais simultaneamente.

2. Análise Matemática do Tempo Total

- O HTML base e o objeto reusado na conexão 1 custam $2\ \mathrm{RTT} + 1\ \mathrm{RTT}$.

- A nova conexão paralela aberta para o objeto 2 custa $2\ \mathrm{RTT}$.
- O tempo total da fase de objetos é o máximo entre as vias paralelas:

$$T_{\text{persistente, paralelo}} = 2\ \mathrm{RTT}\ (\mathrm{HTML}) + \max(1\ \mathrm{RTT}, 2\ \mathrm{RTT}) = 4\ \mathrm{RTT}$$

Tabela Resumo dos Modelos de Conexão HTTP (UFRJ)

MODELO DE CONEXÃO	CONEXÕES TCP ABERTAS	FÓRMULA DO TEMPO TOTAL (PARA N OBJETOS)	VANTAGEM PRINCIPAL	DESVANTAGEM PRINCIPAL
Não Persistente Sem Paralelismo	$n+1$ conexões (1 por vez)	$2(n+1)\ \mathrm{RTT}$	Liberação rápida de recursos no servidor.	Altíssima latência (paga RTT TCP para tudo).
Não Persistente Com Paralelismo	$n+1$ conexões (várias juntas)	$4\ \mathrm{RTT}$	Carregamento muito rápido.	Sobrecarga severa de CPU/sockets no servidor.
Persistente Sem Paralelismo	1 única conexão TCP	$(n+2)\ \mathrm{RTT}$	Economiza RTTs de TCP e recursos no servidor.	Sofre com Head-of-Line (HOL) Blocking.
Persistente Com Paralelismo	Múltiplas conexões mantidas	$4\ \mathrm{RTT}$	Equilíbrio entre paralelismo e reuso de TCP.	Alto número de conexões mantidas ociosas.

---

4. O Bloqueio de Cabeça de Fila (HOL Blocking) e a Evolução do HTTP (HTTP/1.1 $\rightarrow$ HTTP/2 $\rightarrow$ HTTP/3)

Fenômeno de Rede

No HTTP/1.1 com conexão persistente única, as requisições são atendidas na ordem de chegada (FCFS: *First-Come, First-Served*). Se o cliente solicitar um arquivo gigante (ex: um vídeo) seguido por três imagens pequenas, o que acontece?

Modelo Matemático: O Custo do Bloqueio HOL (UFRJ / Kurose)

1. Aplicação do Cálculo

Suponha que um objeto grande O_1 leve **10 unidades de tempo** para ser transmitido e três objetos pequenos (O_2, O_3, O_4) levem **1 unidade de tempo** cada.

- **Cenário A: Atendimento FCFS (Objeto Grande Primeiro - HTTP/1.1)**
  - Instante de conclusão de O_1: $t = 10$.
  - Instante de conclusão de O_2: $t = 10 + 1 = 11$.
  - Instante de conclusão de O_3: $t = 11 + 1 = 12$.
  - Instante de conclusão de O_4: $t = 12 + 1 = 13$.
  - **Tempo Médio de Conclusão Percebido:**

$$\bar{T}_{\text{HOL}} = \frac{10 + 11 + 12 + 13}{4} = \frac{46}{4} = 11{,}5 \text{ unidades}$$

- **Cenário B: Atendimento Reordenado/Intercalado (Objetos Pequenos Primeiro)**
  - Instante de conclusão de O_2: $t = 1$.
  - Instante de conclusão de O_3: $t = 2$.
  - Instante de conclusão de O_4: $t = 3$.
  - Instante de conclusão de O_1: $t = 3 + 10 = 13$.
  - **Tempo Médio de Conclusão Percebido:**

$$\bar{T}_{\text{otimizado}} = \frac{1 + 2 + 3 + 13}{4} = \frac{19}{4} = 4{,}75 \text{ unidades}$$

2. Interpretação em Termos de Redes

O arquivo grande termina no exato mesmo instante ($t = 13$) em ambos os cenários. No entanto, reordenar/intercalar a transmissão reduz o tempo médio de espera do usuário de **11,5 para 4,75 (uma redução de mais de 58%)** sem adicionar 1 bps de largura de banda à rede!

Solução Tecnológica nas Versões do HTTP

1. **HTTP/2 (Padronizado em 2015):**
   - **Intercalação de Quadros (*Frame Interleaving*):** Divide os objetos em quadros pequenos e intercala sua transmissão na mesma conexão TCP. O vídeo grande não bloqueia mais as imagens pequenas.
   - **Problema Residual:** Como o HTTP/2 ainda roda sobre uma **única conexão TCP**, se um único pacote for perdido na rede, o TCP paralisa a entrega de **todos os quadros** até retransmitir o segmento perdido (Bloqueio HOL no nível de Transporte).
2. **HTTP/3 (Padronizado via QUIC):**
   - Substitui o TCP subjacente pelo protocolo **QUIC (que opera sobre UDP)**.
   - O QUIC implementa fluxos de dados independentes e criptografados. Se um pacote de uma imagem for perdido, apenas aquele fluxo sofre retransmissão; todos os outros fluxos continuam fluindo sem nenhum bloqueio!

5. Caches Web (*Proxy Servers*) e o GET Condicional

Fenômeno de Rede

Como reduzir a latência percebida pelo usuário e evitar o estrangulamento de um enlace de acesso saturado sem precisar gastar fortunas aumentando a largura de banda física?

Conceito

Um **Cache Web (Servidor Proxy)** é um nó de rede instalado na LAN local que atende a requisições HTTP em nome do servidor de origem.

- **Dualidade Papel:** O cache age simultaneamente como **servidor** (para os navegadores locais) e como **cliente** (em relação aos servidores de origem na Internet).

Unificação Matemática: O Impacto do Cache no Atraso Médio (Notas UFRJ)

1. Aplicação do Cálculo

Considere uma rede institucional onde a taxa de acerto do cache (*Hit Rate*) é $p_{\text{hit}} = 0{,}40$ (40% das requisições são servidas localmente em $T_{\text{cache}} = 10\ \text{ms}$). As 60% restantes ($p_{\text{miss}} = 0{,}60$) precisam atravessar a Internet e sofrem um atraso total de $T_{\text{Internet}} = 2{,}01\ \text{s}$.

O **Atraso Médio Total de Resposta** é a média ponderada:

$$T_{\text{médio}} = p_{\text{hit}} \cdot T_{\text{cache}} + (1 - p_{\text{hit}}) \cdot T_{\text{Internet}}$$

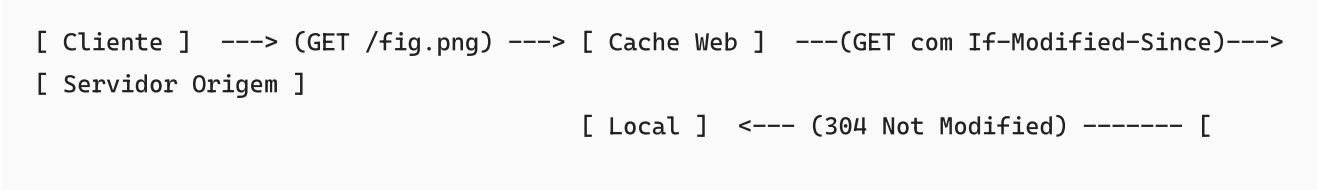
$$T_{\text{médio}} = 0{,}40 \cdot (0{,}010\ \text{s}) + 0{,}60 \cdot (2{,}010\ \text{s}) = 0{,}004 + 1{,}206 = 1{,}21\ \text{segundo}$$

2. Interpretação em Termos de Redes

Além de cortar a latência média de mais de 2 segundos para 1,21 segundo, o cache reduz a carga no enlace de acesso em 40% ($a_{\text{efetiva}} = 0{,}60 \cdot a$). Isso retira a intensidade de tráfego I da zona de saturação (onde $I \to 1$) e estabiliza as filas do roteador.

O Mecanismo do GET Condicional (`304 Not Modified`)

Para evitar que o cache entregue uma cópia local desatualizada de um arquivo que sofreu alterações no servidor de origem, o HTTP utiliza o **GET Condicional**:



```
 Servidor Origem ]
[ Cliente ]  <--- (200 OK + Dados) -- [ Cache Web ]
```

1. Quando o cache armazena um objeto, ele salva a data enviada no cabeçalho `Last-Modified:` do servidor.
2. Quando outro cliente pede o mesmo objeto, o cache envia uma requisição de validação contendo o cabeçalho: `If-Modified-Since: <Data da Cópia Local>`
3. **Se o objeto NÃO foi modificado:** O servidor responde com o código de status `304 Not Modified` com um **corpo de mensagem completamente vazio**.
4. **Resultado:** Economiza-se 100% da largura de banda do arquivo e o cache entrega imediatamente a sua cópia local.

2.4 DNS: O Serviço de Diretório da Internet (*Domain Name System*)

2.4.1 O que é o DNS e Por que Roda na Borda da Rede?

Fenômeno de Rede

Os seres humanos preferem identificar recursos na rede usando nomes mnemônicos e alfanuméricos fáceis de memorizar (como `www.ufrj.br`). No entanto, os roteadores no núcleo da rede processam pacotes examinando endereços IP numéricos de tamanho fixo (como `146.164.220.1`).

Conceito

O **DNS (*Domain Name System*)** é um banco de dados distribuído e hierárquico que provê a **tradução de nomes de domínio em endereços IP** (e vice-versa).

A Filosofia da Arquitetura do DNS

O DNS é um protocolo da **Camada de Aplicação**. Por que a função essencial de traduzir endereços de rede foi projetada na camada de aplicação e não dentro do núcleo da rede (na camada IP)?

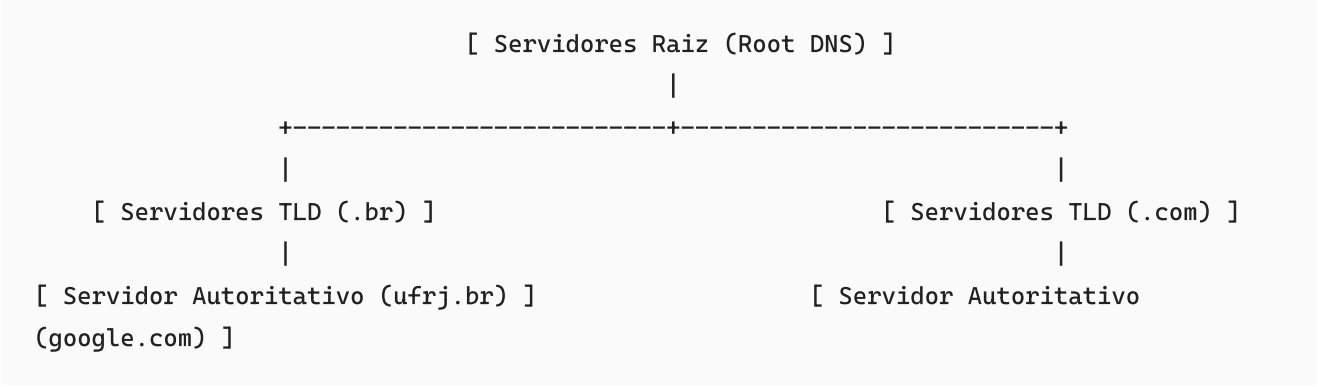
- **Princípio de Projeto da Internet (*End-to-End Principle*):** Manter o núcleo da rede (*routers*) o mais simples, rápido e enxuto possível, deixando a complexidade do banco de dados de nomes para os sistemas finais na borda (*edge*).

2.4.2 Arquitetura Hierárquica e Distribuída do DNS

Se o DNS fosse implementado como um **único servidor centralizado** no mundo, o sistema sofreria com:

1. **Ponto Único de Falha:** Se o servidor caísse, toda a Internet global parava.
2. **Volume Insuportável de Tráfego:** Bilhões de requisições por segundo congestionariam o enlace desse servidor.
3. **Distância Geográfica:** Consultas vindas do outro lado do mundo sofreriam com altíssimos atrasos de propagação.

Por isso, o DNS é estruturado como uma **árvore hierárquica distribuída**:



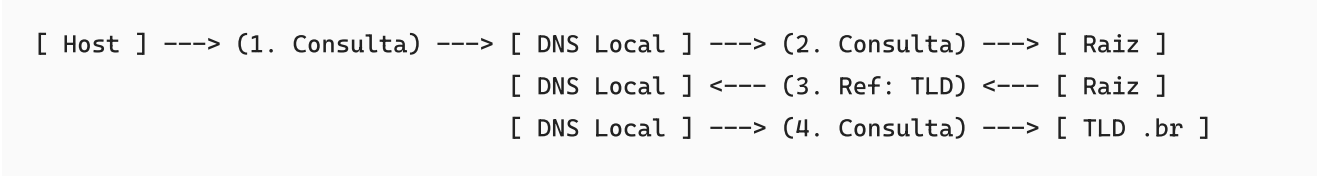
1. **Servidores Raiz (*Root DNS Servers*):** Existem 13 endereços IP lógicos de servidores raiz no mundo (replicados geograficamente por centenas de servidores físicos usando *Anycast*). Eles fornecem os endereços IP dos servidores TLD.
2. **Servidores de Domínio de Nível Superior (*Top-Level Domain - TLD*):** Responsáveis por domínios genéricos (`.com` , `.org` , `.edu`) e domínios de código de país (`.br` , `.uk` , `.fr`).
3. **Servidores Autoritativos (*Authoritative DNS Servers*):** Mantidos por organizações ou provedores. Contêm os registros DNS oficiais que mapeiam os nomes de hospedeiros daquela organização para seus IPs.
4. **Servidor DNS Local (*Local Name Server / Resolver*):** Não pertence estritamente à hierarquia, mas é fundamental. Cada ISP (ou rede universitária) possui um servidor DNS local. Quando um host faz uma consulta DNS, a requisição vai primeiramente para o seu DNS local, que atua como um intermediário/proxy resolvendo a cadeia.

2.4.3 Resolução de Nomes e Análise de RTTs: Iterativa vs. Recursiva (Kurose + UFRJ)

Fenômeno de Rede

Quando seu computador quer acessar `www.ufrj.br` , como o Servidor DNS Local descobre o IP final caso não o tenha em memória?

Exemplo com 4 RTTs em Resolução Iterativa.



```
                                [ DNS Local ] <--- (5. Ref: Auth) <-- [ TLD .br ]
                                [ DNS Local ] ---> (6. Consulta) ---> [ Autoritativo
ufrj.br ]
                                [ DNS Local ] <--- (7. Resposta) <--- [ Autoritativo
ufrj.br ]
[ Host ] <--- (8. IP Final) ---- [ DNS Local ]
```

Unificação Matemática: Modelo de Latência por RTTs na Resolução DNS

1. Resolução Iterativa (Padrão da Internet)

Na consulta **iterativa**, o servidor consultado responde com o endereço do próximo servidor da hierarquia que deve ser consultado (*"Eu não sei o IP, mas pergunte ao servidor X"*).

- **Análise de RTTs:**
  - Host $\rightarrow$ DNS Local: **1 RTT** (geralmente na rede local).
  - DNS Local $\rightarrow$ Servidor Raiz: **1 RTT**.
  - DNS Local $\rightarrow$ Servidor TLD (`.br`): **1 RTT**.
  - DNS Local $\rightarrow$ Servidor Autoritativo (`ufrj.br`): **1 RTT**.
- **Atraso Total sem Cache:**

$$T_{\text{iterativo}} = \text{RTT}_{\text{local}} + \text{RTT}_{\text{raiz}} + \text{RTT}_{\text{TLD}} + \text{RTT}_{\text{auth}} \approx 4\,\text{RTTs}$$

2. Resolução Recursiva

Na consulta **recursiva**, o nó consultado assume a responsabilidade de contatar o nó seguinte e só devolve a resposta quando obtiver o IP final.

- **Carga:** Sobrecarrega os servidores dos níveis superiores da hierarquia (especialmente os servidores Raiz), e por isso é **desabilitada por padrão** nos servidores raiz e TLDs por questões de segurança e desempenho.

3. Ponto-chave de prova: por que ITERATIVO é melhor para POPULAR o cache do servidor local

Essa é uma pergunta que aparece repetidamente nas provas antigas (2026/1 Q1d, 2025/2 Q4a): **entre o host final e o servidor DNS local, sempre se usa resolução recursiva** (o host tem poucos recursos e delega tudo). A pergunta interessante é sobre a etapa seguinte: **entre o servidor local e o resto da hierarquia (raiz/TLD/autoritativo)**, qual abordagem é melhor?

- Se o servidor local usar **recursiva** nessa etapa (delegando a um outro resolver), ele **só recebe a resposta final** — não vê nem armazena nenhuma informação intermediária.
- Se o servidor local usar **iterativa**, ele mesmo vai "saltando" de servidor em servidor, e **em cada resposta parcial** ("pergunte ao servidor X") ele pode **armazenar em cache** aquele registro (por

exemplo, o endereço do servidor TLD `.br`, ou o servidor NS autoritativo de `ufrj.br`). Assim, na **próxima** consulta a um domínio diferente mas sob o mesmo TLD, o servidor local **já sabe** para qual TLD ir, sem precisar consultar a raiz de novo.

Conclusão para a prova: a abordagem **iterativa favorece mais o preenchimento do cache do servidor local**, porque ele enxerga e armazena cada passo intermediário da resolução — a recursiva "esconde" essa informação intermediária dentro do outro resolver.

2.4.4 Registros de Recursos (*Resource Records - RRs*)

O banco de dados do DNS armazena suas informações em registros de quatro campos no formato:

$$(\text{Name}, \text{Value}, \text{Type}, \text{TTL})$$

O campo **Type** define o significado dos campos `Name` e `Value`:

1. **Tipo A (*Address*):** Mapeia um nome de host para um endereço IPv4.
   - `(aulas.ufrj.br, 146.164.220.1, A, 86400)`
2. **Tipo NS (*Name Server*):** Especifica o nome do servidor DNS autoritativo responsável pelo domínio.
   - `(ufrj.br, dns.ufrj.br, NS, 86400)`
3. **Tipo CNAME (*Canonical Name*):** Mapeia um alias/apelido para seu nome canônico (verdadeiro).
   - `(server1.ufrj.br, webserver-prod-01.ufrj.br, CNAME, 86400)`
4. **Tipo MX (*Mail Exchange*):** Mapeia o domínio para o nome do servidor de correio responsável por receber e-mails daquele domínio.
   - `(ufrj.br, mail.ufrj.br, MX, 86400)`

2.4.5 Caching DNS e a Escolha do Transporte: UDP vs. TCP (UFRJ)

Caching DNS

Para reduzir drasticamente o tempo de resposta e o tráfego nos servidores Raiz/TLD, os servidores DNS locais armazenam em **cache** os mapeamentos aprendidos.

- O campo **TTL (*Time to Live*)** especifica após quantos segundos a entrada expira e deve ser descartada do cache para garantir a consistência de dados alterados.

Unificação Matemática: Por que o DNS usa UDP por padrão e TCP como Exceção? (Tema Chave de Prova)

1. Uso do UDP (Porta 53) — Consultas Padrão

- **Fenômeno:** Uma consulta DNS típica (pergunta por um IP) exige uma mensagem pequena de requisição e uma resposta enxuta.
- **Análise de Latência:**
  - Se o DNS utilizasse **TCP**, cada tradução de nome exigiria o *handshake* de 3 vias do TCP (1 RTT) antes mesmo de enviar a pergunta DNS. A resolução DNS sem cache custaria no mínimo $2 \times 4 = 8\ \mathrm{RTTs}$!
  - Com **UDP**, o cliente envia o datagrama direto. A consulta custa apenas **1 RTT por salto**, tornando a navegação Web instantânea.
- **Economia de Recursos:** Servidores DNS recebem bilhões de requisições. Usar UDP evita manter blocos de controle de conexão TCP (*Sockets*) em memória no servidor.

2. Uso do TCP (Porta 53) — Exceção / Transferência de Zona

- **Fenômeno:** Quando a resposta DNS excede o tamanho máximo de um datagrama UDP padrão (**512 bytes**) ou quando dois servidores autoritativos sincronizam todo o seu banco de dados (**Transferência de Zona / *Zone Transfer***).
- **Por que mudar para TCP?** Transferir bancos de dados inteiros exige a garantia de entrega confiável, sem perdas, sem corrupção e com ordenação correta, justificando o uso do TCP. Vamos concluir o **Capítulo 2: Camada de Aplicação**, cobrindo as Seções **2.5 (Distribuição P2P)**, **2.6 (Streaming de Vídeo DASH e CDNs)** e **2.7 (Programação com Sockets)**.

Sempre que a modelagem matemática e os exercícios de prova da UFRJ aprofundarem o texto do Kurose & Ross, unificaremos o conteúdo através da nossa cadeia analítica (**fenômeno de rede → conceito → modelo → matemática → cálculo → interpretação**).

2.5 Distribuição de Arquivos P2P e o Protocolo BitTorrent

---

1. O Fenômeno da Distribuição Massiva de Arquivos

Fenômeno de Rede

Quando uma empresa lança uma atualização de sistema operacional ou um vídeo em alta definição de F **bits** e N **usuários** tentam baixá-lo ao mesmo tempo, como a infraestrutura de rede responde?

Conceito: Arquitetura Cliente-Servidor vs. P2P

- **Na Arquitetura Cliente-Servidor:** O servidor é a única fonte geradora de bits. Conforme o número de clientes N cresce, a taxa de upload do servidor (u_s) torna-se um gargalo severo.
- **Na Arquitetura Peer-to-Peer (P2P):** Os sistemas finais (**pares** ou *peers*) atuam simultaneamente como clientes (consumindo bits) e servidores (redistribuindo bits que já baixaram). O sistema

possui **autoescalabilidade (*self-scalability*)**: cada novo par traz uma nova carga de consumo, mas também adiciona capacidade de upload ao sistema.

---

2. Unificação Matemática: Modelo do Tempo Mínimo de Distribuição (Kurose + UFRJ)

Para comparar quantitativamente o tempo necessário para distribuir um arquivo de tamanho F para N clientes entre as duas arquiteturas, define-se o modelo sob as seguintes variáveis:

- F: Tamanho do arquivo a ser distribuído (em **bits**).
- N: Número de clientes/pares que desejam obter a cópia do arquivo.
- u_s: Taxa de upload do servidor de origem (em **bits/s**).
- u_i: Taxa de upload do i-ésimo par (em **bits/s**).
- d_i: Taxa de download do i-ésimo par (em **bits/s**).
- $d_{\min} = \min d_1, d_2, \ldots, d_N$: Taxa de download do par mais lento da rede.

Modelo Matemático 1: Tempo de Distribuição na Arquitetura Cliente-Servidor (D_{CS})

1. Dedução dos Limites Inferiores

Nenhum par ajuda a redistribuir o arquivo. O tempo total é limitado por dois gargalos físicos:

1. **Gargalo no Servidor:** O servidor precisa enviar N cópias completas de F bits, enviando um total de $N \cdot F$ bits pela sua interface de upload u_s. O tempo não pode ser menor que $\frac{N \cdot F}{u_s}$.
2. **Gargalo no Cliente Mais Lento:** O cliente com a menor taxa de download ($d_{\min}$) leva no mínimo $\frac{F}{d_{\min}}$ para receber seus próprios F bits.

2. Aplicação da Fórmula

$$D_{\text{CS}} = \max\left\{ \frac{N \cdot F}{u_s},; \frac{F}{d_{\text{min}}} \right\}$$

3. Interpretação em Termos de Redes

Para valores grandes de N, o tempo de distribuição é dominado pelo termo $\frac{N \cdot F}{u_s}$. **O tempo cresce de forma estritamente linear com o número de usuários** N. Se o número de clientes aumentar 1000 vezes, o tempo para distribuir o arquivo também aumentará 1000 vezes!

Modelo Matemático 2: Tempo de Distribuição na Arquitetura P2P (D_{P2P})

1. Dedução dos Limites Inferiores

No P2P, os pares redistribuem pedaços do arquivo entre si. O tempo total é limitado por três restrições físicas:

1. **Envio Inicial do Servidor:** Para que o arquivo entre na comunidade, o servidor precisa injetar cada um dos F bits pelo menos uma vez no enlace. Tempo mínimo: $\frac{F}{u_s}$.
2. **Gargalo de Download no Cliente Lento:** O cliente mais lento ainda precisa baixar seus F bits. Tempo mínimo: $\frac{F}{d_{\min}}$.
3. **Capacidade Agregada de Upload do Sistema:** A rede como um todo precisa entregar um total de $N \cdot F$ bits para os N clientes. A taxa máxima de upload combinada de todo o sistema é a soma do upload do servidor com o upload de todos os N pares ($u_{\text{total}} = u_s + \sum_{i=1}^{N} u_i$). Tempo mínimo: $\frac{N \cdot F}{u_s + \sum_{i=1}^{N} u_i}$.

2. Aplicação da Fórmula

D_{\text{P2P}} = \max\left\{ \frac{F}{u_s},; \frac{F}{d_{\text{min}}},; \frac{N \cdot F}{u_s + \sum_{i=

3. Interpretação em Termos de Redes (Análise com Uploads Iguais $u_i = u$)

Se todos os pares tiverem a mesma taxa de upload $u_i = u$, a capacidade total de upload torna-se $u_s + N \cdot u$. O terceiro termo passa a ser:

$$\frac{N \cdot F}{u_s + N \cdot u}$$

Capítulo 3 (parcial — Seções 3.1 a 3.5): Camada de Transporte — TCP e UDP

> **Nota de escopo:** esta seção cobre exatamente o que foi confirmado como matéria de prova — estrutura e funcionamento básico de TCP e UDP. **Controle de congestionamento (Seções 3.6-3.7) está fora do escopo** e não é tratado aqui.

3.1 Introdução à Camada de Transporte

Fenômeno de Rede

A camada de **rede** (IP) só sabe entregar dados **de um host até outro host**, usando o endereço IP. Mas um host roda **vários processos** simultaneamente (navegador, e-mail, jogo online), todos compartilhando o mesmo IP.

Conceito

A camada de **transporte** é responsável pela "última entrega": pegar os dados que chegaram até o host (via IP) e encaminhá-los para o **processo correto**, usando o número de porta. Isso é o processo de **demultiplexação**. O processo inverso, no lado de quem envia — juntar dados de vários processos diferentes e passar para a camada de rede — é a **multiplexação**.

3.2 Multiplexação e Demultiplexação — Formalização

Como o sistema operacional identifica de quem é cada segmento

Cada segmento de transporte carrega, no seu cabeçalho, no mínimo dois campos: **porta de origem** e **porta de destino**.

Demultiplexação sem conexão (UDP)

Um socket UDP é identificado apenas por uma **dupla**: `(endereço IP de destino, porta de destino)`. Isso significa que **dois segmentos UDP vindos de IPs de origem diferentes, mas com o mesmo IP e porta de destino**, são entregues ao **mesmo socket** — o UDP não diferencia origens diferentes, ele só olha para onde o segmento está indo.

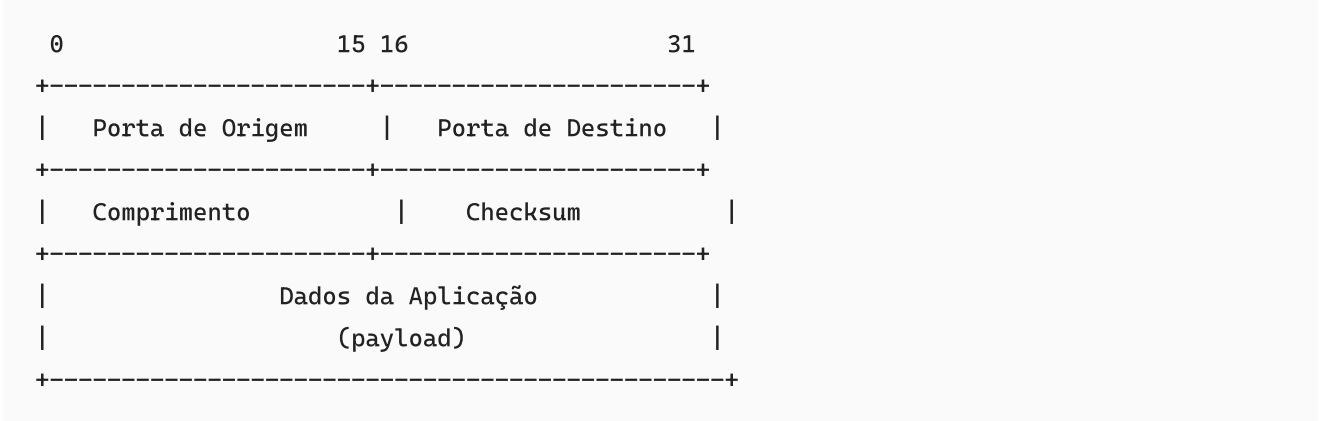
Demultiplexação orientada a conexão (TCP)

Um socket TCP é identificado por uma **quádrupla**: `(IP de origem, porta de origem, IP de destino, porta de destino)`. Isso significa que o sistema operacional consegue diferenciar **conexões diferentes chegando na mesma porta de destino** — por exemplo, dois navegadores diferentes no seu computador, ambos acessando o mesmo site (mesmo IP:porta de destino), geram **quádruplas diferentes** (portas de origem diferentes, escolhidas aleatoriamente pelo sistema operacional para cada conexão nova), e por isso o servidor consegue tratar cada conexão de forma isolada, com seu próprio `connectionSocket` dedicado.

Por isso o servidor Web consegue atender múltiplos clientes simultâneos na mesma porta 80: cada cliente gera uma quádrupla única, mesmo compartilhando IP e porta de destino.

3.3 UDP — Estrutura e Funcionamento

Formato do segmento UDP (cabeçalho de apenas 8 bytes)



- **Porta de origem / destino:** 16 bits cada — usadas na demultiplexação.
- **Comprimento:** tamanho do segmento UDP (cabeçalho + dados), em bytes.

- **Checksum:** verifica se houve erro de bit durante a transmissão — soma de verificação simples, calculada sobre o segmento inteiro (cabeçalho + dados) mais um "pseudo-cabeçalho" com os IPs de origem/destino.

Por que UDP é "leve"

UDP **não** estabelece conexão, **não** garante entrega, **não** garante ordem, e **não** faz controle de fluxo nem de congestionamento. Isso o torna ideal para aplicações que preferem velocidade/simplicidade a garantias (DNS, VoIP, streaming ao vivo, jogos), como já discutimos no Capítulo 2.

3.4 Transferência Confiável de Dados (RDT) — Construindo do Zero

Essa é a base conceitual de como o TCP garante entrega confiável **por cima** de uma rede que, por si só (a camada IP), **não** garante nada. O livro constrói isso incrementalmente, em versões (rdt 1.0, 2.0, 2.1, 2.2, 3.0) — cada uma resolvendo um problema novo que a versão anterior não cobria.

rdt 1.0 — Canal perfeitamente confiável (caso trivial)

Se o canal de comunicação **nunca** perde, corrompe ou atrasa nada, o transmissor simplesmente envia, e o receptor simplesmente recebe. Nenhum mecanismo extra é necessário. Isso é irrealista, mas serve de ponto de partida.

rdt 2.0 — Canal com erros de bit (mas sem perdas)

Agora os bits podem ser **corrompidos** durante a transmissão (mas não perdidos). O receptor precisa de uma forma de **detectar** erros, e o transmissor precisa saber se deve **retransmitir**.

Mecanismos introduzidos:

- **Checksum:** permite ao receptor detectar corrupção.
- **ACK (Acknowledgment):** o receptor envia de volta "recebi certo" quando o pacote chega sem erro.
- **NAK (Negative Acknowledgment):** o receptor envia "recebi errado" quando detecta corrupção via checksum — o transmissor então retransmite o mesmo pacote.

Problema não resolvido por rdt 2.0: e se o **próprio ACK/NAK** for corrompido? O transmissor não saberia se deve retransmitir ou não.

rdt 2.1 — Resolvendo ACK/NAK corrompidos com números de sequência

Solução: adicionar um **número de sequência** de 1 bit (0 ou 1, alternando a cada novo pacote — por isso esse protocolo também é chamado de **protocolo do bit alternante**, *alternating bit protocol*) a cada pacote de dados.

Se o transmissor recebe um ACK/NAK **corrompido** (ilegível), ele simplesmente **retransmite** o **último** pacote enviado (por segurança, já que não sabe se foi bem recebido). O receptor, ao receber um pacote **duplicado** (mesmo número de sequência do último que já recebeu com sucesso), **descarta** os dados (já os processou antes) mas **reenvia o ACK** — para garantir que o transmissor eventualmente saiba que está tudo certo.

Pergunta clássica de prova: *"Quando o emissor recebe um ACK corrompido ou inesperado, o que ele deve fazer?"* — **Resposta:** ele **ignora** esse ACK (não sabe interpretá-lo com confiança) e **aguarda o timeout**; se o timeout expirar sem receber um ACK válido, ele **retransmite** o último pacote enviado.

rdt 2.2 — Versão só com ACKs (sem NAKs)

É possível eliminar o NAK completamente: em vez de mandar "NAK, está corrompido", o receptor manda um **ACK duplicado** (reenvia o ACK do **último pacote correto recebido**). Quando o transmissor recebe **dois ACKs iguais em seguida** (um ACK duplicado), ele interpreta isso como um sinal implícito de que o próximo pacote esperado não chegou corretamente, e retransmite.

Pergunta clássica de prova: *"É possível implementar comunicação confiável usando apenas ACKs, sem NACKs?"* — **Resposta: Sim.** ACKs duplicados cumprem exatamente o mesmo papel informativo que um NAK explícito.

rdt 3.0 — Canal com erros E perdas (o protocolo "Stop-and-Wait" completo)

Agora, além de corrupção, o canal também pode **perder** pacotes ou ACKs inteiros (não chegam de jeito nenhum). Como o transmissor detecta uma perda, se nunca vai receber nenhum ACK/NAK sobre ela?

Solução: temporizador (timeout). O transmissor, toda vez que envia um pacote, **inicia um timer**. Se o ACK correspondente não chegar antes do timer expirar, o transmissor **assume que houve perda** (do pacote ou do ACK) e **retransmite**.

O dilema do valor do timeout (pergunta recorrente de prova):

- **Timeout muito pequeno:** o transmissor retransmite **antes da hora**, mesmo quando o pacote/ACK só está **atrasado** (ainda a caminho) — gerando retransmissões desnecessárias e desperdício de banda.
- **Timeout muito grande:** em caso de perda real, o transmissor **demora demais** para perceber e retransmitir — desperdiçando tempo ocioso e reduzindo a vazão efetiva.
- **Solução do TCP (avançado, mas cobrado):** o timeout é calculado **dinamicamente**, baseado numa **média móvel ponderada do RTT observado** (EstimatedRTT) mais uma margem de segurança proporcional à **variação** do RTT (DevRTT) — não é um valor fixo.

O que o receptor faz ao receber um pacote com número de sequência inesperado (pergunta recorrente): o receptor **descarta silenciosamente** o pacote errado (fora de ordem) e **reenvia o ACK do último pacote recebido corretamente em ordem** — isso "empurra" o transmissor a perceber, via ACKs duplicados ou timeout, que precisa retransmitir o que falta.

Utilização (eficiência) do protocolo Stop-and-Wait

Como o transmissor **espera** o ACK antes de enviar o próximo pacote, o enlace fica **ocioso** durante boa parte do tempo — especialmente se o RTT for grande comparado ao tempo de transmissão. A **utilização** U é definida como a fração de tempo em que o enlace está efetivamente ocupado transmitindo dados:

$$U = \frac{L/R}{L/R + RTT}$$

onde L/R é o tempo de transmissão de um pacote de dados, e RTT é o tempo de propagação de ida e volta (assumindo tempo de transmissão do ACK desprezível). **Isso é literalmente o mesmo cálculo que já vimos no gabarito de 2025/2 (Questão 1d):** com $S = 1$ (só 1 número de sequência de espaço, janela $J = 1$), o protocolo se comporta exatamente como stop-and-wait, e a utilização é essa fração — a mesma fórmula de "tempo útil ÷ tempo total do ciclo" que já vimos em outros contextos (como intensidade de tráfego).

Interpretação: para RTTs grandes (enlaces de longa distância, como satélite), a utilização do stop-and-wait pode ser **catastroficamente baixa** — isso motiva a próxima técnica.

Pipelining — Go-Back-N (GBN) e Selective Repeat (SR)

O problema do stop-and-wait: só um pacote "no ar" por vez, desperdiçando o enlace. **Solução: pipelining** — permitir múltiplos pacotes "em trânsito" simultaneamente, sem esperar confirmação individual de cada um, usando uma **janela** de tamanho J (o transmissor pode ter até J pacotes não confirmados ao mesmo tempo).

Go-Back-N (GBN)

- O **receptor** é simples: só aceita pacotes **em ordem estrita**. Qualquer pacote fora de ordem é **descartado** (mesmo que não esteja corrompido!), e o receptor reenvia um ACK **cumulativo** do último pacote recebido em ordem.
- **ACK cumulativo:** confirma "recebi tudo certinho até o número X" (não confirma pacotes individuais fora de ordem).
- Se o transmissor detecta perda (via timeout do pacote mais antigo não confirmado), ele **retransmite todos os pacotes da janela a partir daquele ponto** — daí o nome "volte N" (Go-Back-N): mesmo pacotes que já tinham chegado corretamente ao receptor (mas fora de ordem) são reenviados, porque o receptor os descartou.
- **Vantagem:** receptor simples (não precisa de buffer para reordenar). **Desvantagem:** desperdiça banda retransmitindo pacotes que talvez já tivessem chegado bem.

Números de sequência necessários: com janela de tamanho J, o espaço de números de sequência precisa ter pelo menos **J+1** valores diferentes (não pode ser igual a J), para o receptor conseguir diferenciar um pacote **novo** de uma **retransmissão antiga** que ainda esteja circulando na rede.

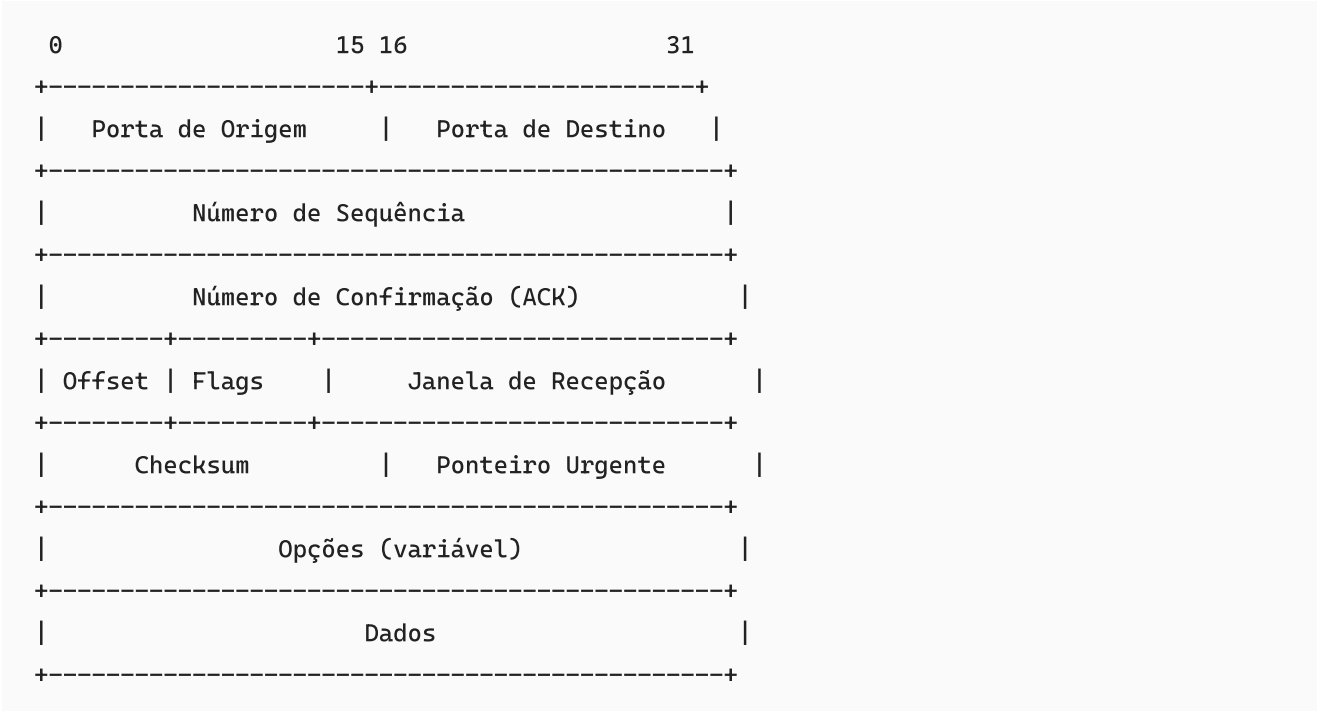
Selective Repeat (SR)

- O **receptor** é mais sofisticado: aceita e **armazena em buffer** pacotes fora de ordem (desde que estejam dentro da janela de recepção), enviando um **ACK individual** para cada pacote correto recebido — mesmo os fora de ordem.
- Quando o pacote faltante finalmente chega, o receptor consegue **reordenar** e entregar tudo em sequência para a camada de aplicação, sem precisar que o transmissor reenvie pacotes que **já** chegaram corretamente.
- **Retransmissão:** o transmissor só reenvia o(s) pacote(s) **especificamente** não confirmado(s) — daí "repetição seletiva".
- **Vantagem:** usa a banda de forma muito mais eficiente (menos retransmissões desnecessárias). **Desvantagem:** receptor mais complexo (precisa de buffer de reordenação); e o espaço de números de sequência precisa ser **pelo menos o dobro do tamanho da janela (2J)**, porque tanto o transmissor quanto o receptor podem estar com visões parcialmente diferentes de quais pacotes já foram confirmados.

Conexão com o gabarito 2025/2 (Questão 1e): o "menor S" necessário para não confundir retransmissão com nova emissão depende de **o receptor descartar ou não pacotes fora de ordem**: se ele descarta (GBN), bastam **J** números de sequência adicionais (S≥J); se ele aceita fora de ordem (SR, sem descartar), são necessários **2J**.

3.5 TCP — Estrutura e Funcionamento Básico

Formato do segmento TCP (cabeçalho de 20 bytes, sem opções)



Campos-chave que mais caem em prova:

- **Número de sequência:** identifica a posição, em **bytes**, do primeiro byte de dados daquele segmento **dentro do fluxo de bytes** da conexão (TCP não numera *pacotes*, numera *bytes*! Isso é diferente do RDT genérico que vimos acima, que numera pacotes).
- **Número de confirmação (ACK):** número do **próximo byte esperado** pelo receptor (ACK **cumulativo** — confirma tudo até ali, em ordem).
- **Janela de recepção (Receive Window):** usada para **controle de fluxo** (ver abaixo).
- **Flags:** incluem SYN (sincronizar/abrir conexão), ACK (confirmação válida), FIN (encerrar conexão), RST (resetar conexão abruptamente).

Gerenciamento de conexão: abertura (Three-Way Handshake)

Já vimos isso no contexto do HTTP, mas formalizando como conceito do TCP em si:

1. **Cliente → Servidor: segmento SYN** (com um número de sequência inicial escolhido aleatoriamente pelo cliente, `client_isn`)
2. **Servidor → Cliente: segmento SYN-ACK** (confirma o SYN do cliente com `ACK = client_isn+1`, e envia seu próprio número de sequência inicial `server_isn`)
3. **Cliente → Servidor: segmento ACK** (confirma o SYN do servidor com `ACK = server_isn+1`) — a partir daqui, dados já podem ser enviados junto com esse terceiro segmento

Por que 3 vias e não 2? Com apenas 2 vias (SYN, depois SYN-ACK), o **servidor** não teria confirmação de que o cliente realmente recebeu seu SYN-ACK — ele alocaria recursos (buffers, número de sequência inicial) para uma conexão que talvez o cliente nem tenha completado do outro lado. O terceiro segmento (ACK) garante que **ambos os lados** confirmaram a troca de números de sequência iniciais antes de qualquer dado real ser trocado.

Pergunta-armadilha clássica (aparece no gabarito 2022/1, Q4c): *"O three-way-handshake estabelece um circuito (circuit switch) half-duplex?"* — **FALSO.** O handshake estabelece uma conexão **lógica** (um estado compartilhado entre os dois lados, com números de sequência sincronizados), **não** uma reserva física de recursos como em comutação de circuitos — os pacotes TCP continuam trafegando por comutação de **pacotes**, disputando enlaces normalmente. Além disso, a conexão TCP é **full-duplex** (dados fluem nos dois sentidos simultaneamente), não half-duplex.

Gerenciamento de conexão: encerramento (Four-Way)

1. **Cliente → Servidor:** segmento FIN
2. **Servidor → Cliente:** ACK do FIN (o servidor pode ainda ter dados a enviar)
3. **Servidor → Cliente:** segmento FIN próprio, quando terminar de enviar tudo
4. **Cliente → Servidor:** ACK final

O encerramento precisa de **4** mensagens (não 3) porque a conexão é full-duplex — cada lado precisa encerrar seu **próprio** fluxo de envio de forma independente.

Controle de Fluxo (Flow Control) — não confundir com controle de congestionamento!

Ponto de atenção crítico (pegadinha recorrente — gabarito 2022/1, Q4b): controle de fluxo e controle de congestionamento **não são a mesma coisa**, e a afirmação "controle de congestionamento evita gargalo no receptor, controle de fluxo evita gargalo no emissor" está **invertida e errada**.

- **Controle de FLUXO:** evita que o **transmissor sobrecarregue o BUFFER do RECEPTOR** — se o receptor for lento para processar os dados (aplicação lenta lendo do socket), seu buffer pode encher. O receptor anuncia, no campo **janela de recepção** do cabeçalho TCP, **quanto espaço livre ainda tem** no seu buffer — e o transmissor nunca envia mais dados do que esse valor permite.
- **Controle de CONGESTIONAMENTO** (fora do escopo desta prova, mas para não confundir): evita que o transmissor sobrecarregue a **rede** (roteadores/enlaces no meio do caminho), não o receptor.

Resumindo a tabela de confusão: controle de fluxo = problema é o **receptor** (fim a fim, dentro do TCP); controle de congestionamento = problema é a **rede** (os enlaces/roteadores no meio do caminho). Nenhum dos dois tem relação com comutação de circuitos ou de pacotes como "abordagem" — ambos operam dentro de uma rede de comutação de **pacotes**.

PARTE II — PRÁTICA: QUESTÕES DE PROVAS RESOLVIDAS PASSO A PASSO

Questão A — Dedução completa da Lei de Little + fila M/M/1 (tema mais recorrente de todas as provas)

(Baseado em 2025/1 Q1 e no "preencha as lacunas" de 2025/2 Q3)

Enunciado resumido: Explique a Lei de Little e sua aplicação em filas de roteadores. Deduza $E(N) = I/(1-I)$. Calcule para $L = 1000$ bits, $a = 500$ pacotes/s, $R = 1$ Mbps.

Passo 1 — O que é a Lei de Little

A Lei de Little é uma relação de **conservação**, válida para **qualquer** sistema de fila em equilíbrio, independente da distribuição de chegadas ou de serviço:

$$N = a \cdot T$$

onde N é o número médio de itens no sistema, a é a taxa média de chegada, e T é o tempo médio que cada item passa no sistema. **Intuição:** numa janela de tempo longa t, chegam aproximadamente at pacotes; se cada um passa T segundos no sistema, o total de "pacote-segundos" acumulados é

$(at) \cdot T$. Por outro lado, se em média há N pacotes no sistema a cada instante, a mesma área acumulada é $N \cdot t$. Igualando: $Nt = (at)T \implies N = aT$.

Passo 2 — Por que ela se aplica a filas de roteadores

Um roteador com um enlace de saída é modelado como um sistema de fila com 1 servidor: pacotes chegam a uma taxa a, esperam se o enlace estiver ocupado, e são "servidos" (transmitidos) a uma taxa $\mu = R/L$. A Lei de Little se aplica diretamente a esse sistema, tanto ao sistema completo (fila+servidor) quanto só à fila.

Passo 3 — Deduzindo $N = I/(1-I)$ (modelo M/M/1)

Um pacote que chega encontra, em média, N_q pacotes na fila (cada um exigindo um tempo de serviço completo $S = L/R$ até ele) e o servidor ocupado com probabilidade I (contribuindo, por falta de memória da exponencial, com um tempo residual médio igual ao tempo de serviço completo S). Logo, o tempo médio de espera na fila:

$$W = (N_q + I) \cdot S$$

Pela Lei de Little aplicada só à fila: $N_q = a \cdot W$. Substituindo:

$$W = (aW + I) \cdot S = aWS + IS$$

Como $aS = a \cdot L/R = I$:

$$W = IW + IS \implies W(1-I) = IS \implies W = \frac{IS}{1-I}$$

O tempo total no sistema: $T = W + S = \dfrac{IS}{1-I} + S = \dfrac{IS + S(1-I)}{1-I} = \dfrac{S}{1-I}$

Pela Lei de Little ao sistema completo: $N = aT = a \cdot \dfrac{S}{1-I} = \dfrac{aS}{1-I}$. Como $aS = I$:

$$\boxed{N = \frac{I}{1-I}}$$

Passo 4 — Número médio no buffer (só fila, sem o pacote em serviço)

$$N_q = aW = a \cdot \frac{IS}{1-I} = \frac{I \cdot (aS)}{1-I} = \frac{I^2}{1-I}$$

Passo 5 — Aplicando os números

$L = 1000$ bits, $a = 500$ pacotes/s, $R = 1\ \text{Mbps} = 10^6$ bits/s.

$$I = \frac{aL}{R} = \frac{500 \times 1000}{10^6} = \frac{500,000}{1,000,000} = 0{,}5$$

$$N = \frac{I}{1-I} = \frac{0{,}5}{0{,}5} = \boxed{1 \text{ pacote (em média, no sistema)}}$$

$$N_q = \frac{I^2}{1-I} = \frac{0{,}25}{0{,}5} = \boxed{0{,}5 \text{ pacote (em média, só no buffer)}}$$

Questão B — Cache antes do roteador, com redução de capacidade (Q1 completa, prova 2026/1)

Roteador modelado como fila; $I < 1$; $N = I/(1-I)$; $N = aT$ (Little).

(a) Cache reduz a chegada pela metade: $a' = a/2$. Encontre N' em função de N, e mostre $N' < 1$.

A nova intensidade de tráfego: $I' = \dfrac{a'L}{R} = \dfrac{(a/2)L}{R} = \dfrac{I}{2}$

$$N' = \frac{I'}{1-I'} = \frac{I/2}{1-I/2} = \frac{I}{2-I}$$

Expressando em função de N: de $N = I/(1-I)$, isolamos $I = N/(1+N)$. Substituindo:

$$N' = \frac{N/(1+N)}{2 - N/(1+N)} = \frac{N}{2(1+N) - N} = \frac{N}{N+2}$$

Mostrando que $N' < 1$: como $N \geq 0$, temos $N + 2 > N$, logo $\dfrac{N}{N+2} < 1$ **sempre**, para qualquer $N \geq 0$ original. ■

Interpretação intuitiva: não importa o quão congestionado o sistema **original** estivesse (mesmo com N grande, sistema quase saturado), depois de **cortar a taxa de chegada pela metade**, o número médio de pacotes no roteador **nunca** ultrapassa 1 — o cache "tira" o sistema da zona de explosão não-linear de $N = I/(1-I)$ perto de $I = 1$.

(b) Cache reduz $a \to a/2$ E capacidade reduz $R \to R/2$. Encontre N'' e T''.

$$I'' = \frac{(a/2)L}{R/2} = \frac{aL}{R} = I \quad (\text{os fatores } 1/2 \text{ se cancelam!})$$

$$N'' = \frac{I''}{1-I''} = \frac{I}{1-I} = N$$

Para T'': como $S'' = L/R'' = L/(R/2) = 2L/R = 2S$ (tempo de serviço **dobra**, já que a capacidade caiu pela metade):

$$T'' = \frac{S''}{1-I''} = \frac{2S}{1-I} = 2 \cdot \frac{S}{1-I} = 2T$$

A cache compensou a redução de capacidade? $N'' = N$ significa que o número médio de pacotes **não mudou** — parece que sim, compensou perfeitamente. Mas $T'' = 2T$ mostra que o **tempo** que cada pacote passa no sistema **dobrou**! A cache manteve a "ocupação" do roteador igual, mas cada pacote individual demora o dobro para ser atendido, porque o enlace ficou mais lento. **Não houve compensação real do ponto de vista do usuário** — a percepção de atraso piorou, mesmo com N igual.

(c) Números: $L = 1$ Mbit, $a = 50$ pac/s, $R = 100$ Mbit/s

(i) Sistema original:

$$I = \frac{aL}{R} = \frac{50 \times 1}{100} = 0{,}5 \qquad N = \frac{0{,}5}{0{,}5} = 1 \qquad T = \frac{N}{a} = \frac{1}{50} = 0{,}02\text{s} = 20\text{ms}$$

(ii) Cache ($a' = 25$):

$$I = \frac{25 \times 1}{100} = 0{,}25 \qquad N = \frac{0{,}25}{0{,}75} = 0{,}333 \qquad T = \frac{0{,}333}{25} = 0{,}0133\text{s} = 13{,}3\text{ms}$$

(iii) Banda dobrada ($R' = 200$):

$$I = \frac{50 \times 1}{200} = 0{,}25 \qquad N = \frac{0{,}25}{0{,}75} = 0{,}333 \qquad T = \frac{0{,}333}{50} = 0{,}00667\text{s} = 6{,}67\text{ms}$$

(iv) Cache + banda reduzida ($a' = 25$, $R' = 50$):

$$I = \frac{25 \times 1}{50} = 0{,}5 \qquad N = \frac{0{,}5}{0{,}5} = 1 \qquad T = \frac{1}{25} = 0{,}04\text{s} = 40\text{ms}$$

(v) Comparação qualitativa (assumindo hit da cache tem custo desprezível): o cenário **(iii)** dobrar a banda é o **melhor** para o usuário (menor T=6,67ms, sem mexer na taxa de chegada real de misses). O cenário **(ii)**, cache sozinha, também melhora bem (T caiu de 20 para 13,3ms) — e como metade das requisições nem chega a passar pelo roteador (acerto de cache, custo desprezível), o **atraso médio percebido pelo usuário** é ainda menor que os 13,3ms calculados (que só valem para os misses). O cenário **(iv)** é o **pior**: mesmo com a cache ajudando, a redução de capacidade anula o ganho e ainda dobra o T em relação ao original.

(d) DNS recursivo vs. iterativo

Já cobrimos essa explicação completa na seção 2.4.3 acima — **iterativo favorece o preenchimento do cache do servidor local**, porque ele vê e armazena cada resposta intermediária da hierarquia, diferente do recursivo, que só recebe a resposta final pronta.

(e) Espaço médio ocupado pela cache DNS (TTL)

Requisições chegam a taxa λ; hit com probabilidade h; miss com probabilidade $(1-h)$ cria uma nova entrada; cada entrada fica em média D segundos e ocupa b bits.

Passo 1: a taxa de **criação** de novas entradas na cache é a taxa de misses: $\lambda_{miss} = \lambda \cdot (1-h)$.

Passo 2: cada entrada, uma vez criada, permanece por um tempo médio D antes de expirar (TTL). Isso é **exatamente** a estrutura da Lei de Little: "entradas" são como "pacotes no sistema", a "taxa de chegada" é $\lambda(1-h)$, e o "tempo no sistema" é D. Logo, o **número médio de entradas** na cache em um dado instante:

$$N_{entradas} = \lambda(1-h) \cdot D$$

Passo 3: cada entrada ocupa b bits, então o **espaço médio ocupado**:

$$\boxed{\text{Espaço médio} = \lambda(1-h)\cdot D \cdot b \text{ bits}}$$

Questão C — Verdadeiro ou Falso: as pegadinhas mais recorrentes (compiladas de 2026/1, 2025/1, 2024/2, 2022/1)

1. "O TCP é stateless, pois roda sobre uma rede packet switching que não mantém estado." → **FALSO.** TCP **é stateful** — mantém números de sequência, janela, estado da conexão (estabelecida, fechando, etc.) durante toda a vida da conexão. O tipo de comutação da rede subjacente (packet switching) não determina se o protocolo de transporte por cima é stateful ou não — são camadas diferentes, com decisões de design independentes.

2. "O UDP é stateful porque usa checksum." → **FALSO.** Checksum verifica a integridade de **um único datagrama isolado** — não implica guardar nenhuma informação entre datagramas diferentes. UDP continua sem conexão e sem estado.

3. "HTTP é stateful, motivando o uso de cookies." → **FALSO (invertido).** HTTP é **stateless** — e é **por isso** que cookies existem: para **compensar** a ausência nativa de estado.

4. "O núcleo da Internet é baseado em protocolos stateful para garantir confiabilidade." → **FALSO.** O núcleo (roteadores, protocolo IP) é **stateless por design** — cada roteador trata cada datagrama IP de forma independente ("best effort", sem lembrar de pacotes anteriores). A confiabilidade, quando existe, é garantida **fim a fim** (pelo TCP nos hosts extremos), não dentro do núcleo.

5. "Controle de congestionamento evita gargalo no receptor; controle de fluxo evita gargalo no emissor; congestionamento usa circuit switching, fluxo usa packet switching." → **FALSO em tudo.** Invertido: controle de **fluxo** protege o **receptor** (buffer dele); controle de **congestionamento** protege a **rede** (enlaces/roteadores no meio). Nenhum dos dois usa "abordagem circuit switching" — ambos operam dentro de uma rede de comutação de pacotes, apenas ajustando a taxa/janela de envio.

6. "Three-way handshake estabelece um circuito half-duplex." → **FALSO.** Estabelece uma conexão **lógica** (não uma reserva física de recursos como em comutação de circuitos), e a conexão resultante é **full-duplex** (dados nos dois sentidos simultaneamente), não half-duplex.

7. "HFC usa FDMA, sendo portanto circuit switching." → **FALSO.** FDMA é uma **técnica de multiplexação** (divide o espectro em faixas), mas isso **não implica automaticamente** comutação de circuitos — o cabo HFC é compartilhado entre vizinhos de forma que pode envolver disputa/contenção (mais próximo do espírito de packet switching no acesso), mesmo usando FDM para separar canais.

8. "HTTP/2.0 é stateful porque permite priorização de requisições, enquanto HTTP/1.0 é stateless." → **FALSO.** Priorização de requisições é um recurso de **como a conexão TCP é usada** (interleaving), não muda a semântica da camada de aplicação HTTP, que continua **stateless em ambas as versões**.

9. "Basta enviar o cookie na primeira mensagem TCP para ele funcionar." → **FALSO.** O cookie precisa ser enviado pelo **servidor** na resposta HTTP (`Set-Cookie:`), e devolvido pelo **cliente** em

toda requisição HTTP subsequente (`Cookie:`) — não é algo enviado "uma vez só" na conexão TCP; é repetido a cada requisição HTTP, porque HTTP é stateless e cada requisição precisa "reapresentar" o cookie.

10. "O comando close gera tráfego ao fechar sockets TCP e UDP." → PARCIALMENTE FALSO — pegadinha. Fechar um socket **TCP** gera tráfego real (segmentos FIN/ACK do encerramento de 4 vias). Fechar um socket **UDP**, por outro lado, é uma operação **puramente local** (não há conexão estabelecida para "encerrar" com o outro lado) — **não gera nenhum tráfego de rede**.

Questão D — Probabilidade de sucesso e retransmissões com perda por enlace (2022/1, itens d-e)

N enlaces em série, cada um com probabilidade de perda p independente.

(d) Probabilidade de um pacote chegar com sucesso ao cliente:

Para ter sucesso, o pacote precisa **sobreviver a todos os N enlaces**. Como as perdas são independentes, multiplicamos as probabilidades de sucesso individual $(1-p)$ em cada enlace:

$$\boxed{P(\text{sucesso}) = (1-p)^N}$$

(e) Probabilidade de exatamente k retransmissões até o sucesso:

Isso é uma **Distribuição Geométrica**: "k falhas seguidas, depois um sucesso". Seja $q = (1-p)^N$ a probabilidade de sucesso em **uma** tentativa (uma travessia completa dos N enlaces). A probabilidade de exatamente k retransmissões (ou seja, k falhas antes do sucesso, totalizando $k+1$ tentativas) é:

$$\boxed{P(k \text{ retransmissões}) = (1-q)^k \cdot q = \left[1-(1-p)^N\right]^k \cdot (1-p)^N}$$

Atenção à contagem: $k = 0$ significa "acertou de primeira, sem nenhuma retransmissão" — por isso o suporte começa em $k = 0$ aqui (não em 1 como na Geométrica "padrão" que conta *tentativas*, já que aqui a variável conta *retransmissões*, que é tentativas−1).

Questão E — Checksum de 2 bits detectando erro de 1 bit (2022/1, Q3a)

Mensagem original de 2 bits. Quantos bits de checksum detectam corrupção de até 1 bit?

Passo 1 — Ideia do checksum de paridade: para detectar **qualquer** erro de 1 bit, um checksum de **1 bit de paridade** já basta: soma-se (XOR) os bits da mensagem, e esse resultado é o bit de paridade.

Exemplo concreto: mensagem de 2 bits $m_1 m_2$, bit de paridade $p = m_1 \oplus m_2$.

MENSAGEM ORIGINAL ($M_1 M_2$)	PARIDADE P	TRANSMITIDO (3 BITS)
00	0	000
01	1	011
10	1	101
11	0	110

Se **um único bit** (qualquer um dos 3) for invertido por corrupção, o XOR de todos os 3 bits recebidos deixa de bater com 0 (paridade par esperada) — detectando o erro.

O que acontece se o bit de overhead (paridade) for corrompido, e não os dados? A detecção **ainda funciona**: a verificação de paridade no receptor vai acusar inconsistência de qualquer forma, mesmo que o erro esteja no próprio bit de paridade — porque a verificação olha para a **consistência entre todos os 3 bits recebidos**, não sabe (nem precisa saber) qual bit especificamente foi corrompido.

Overhead adicional necessário além do checksum: sim — checksum sozinho só **detecta** erro, não é suficiente para uma transferência confiável completa. É necessário também: **números de sequência** (para diferenciar pacotes novos de retransmissões) e o mecanismo de **ACK/NAK + timeout** (para o transmissor saber que precisa retransmitir).

Questão F — Blocos vs. Fluido no P2P (2022/2) — já resolvida na parte teórica

Ver seção 2.5 acima, "Interpretação em Termos de Redes (Análise com Uploads Iguais)" — mostra que no limite $N \to \infty$ com uploads iguais, $D_{P2P} \to F/u$ (não cresce com N), demonstrando formalmente a autoescalabilidade.

Questão G — GBN: números de sequência possíveis na janela (2025/2, Q1a — dedução completa)

> $J = 1$, $S = 2$ (sequências possíveis: 0, 1, 2). Receptor espera pacote 0. Quais sequências podem estar na janela do emissor?

Passo 1: com $J = 1$, o emissor só pode ter **1 pacote pendente de ACK por vez** — comporta-se, na prática, como stop-and-wait.

Passo 2: se o receptor está esperando o pacote **0**, existem exatamente **dois cenários possíveis** para o que o emissor tem na janela:

- **Cenário 1 (início de novo ciclo):** o emissor já recebeu o ACK do pacote 2 (o anterior ao 0 no ciclo, em aritmética módulo 3) e **já enviou** o novo pacote 0 — nesse caso, a janela do emissor contém

o pacote **0**.

- **Cenário 2 (ACK ainda não chegou):** o receptor já recebeu com sucesso os pacotes 0, 1 e 2 (por isso está esperando o próximo, que seria o 0 do ciclo seguinte), mas o **ACK do pacote 2** ainda está a caminho de volta ao emissor — nesse caso, o emissor **ainda está esperando** a confirmação do pacote **2**, que continua sendo o último pacote na sua janela.

Por que o pacote 1 NÃO pode estar na janela: se o emissor tivesse o pacote 1 pendente, isso significaria que o **último pacote confirmado** pelo emissor foi o 0 — e nesse caso, o receptor já teria recebido o 0 e estaria esperando o **1**, não o 0. Contradição com o enunciado (receptor espera 0).

Resposta: a janela do emissor, no instante t, contém ou o pacote 0, ou o pacote 2 (nunca o 1).